

PYTHON FASTAPI

AI-Powered Full-Stack Application

A complete from-scratch student handbook
CRUD • SQLite • SQLAlchemy • HTML • JavaScript • OpenAI

Project: AI-Powered Employee Management System

BEGINNER FRIENDLY

FULL SOURCE CODE

PRACTICE TASKS

How to Use These Notes

Follow the chapters in order. Type the code yourself instead of copying it blindly. Run and test the application after every important step.

PROJECT GOAL

Create a browser-based Employee Management System that performs CRUD operations and answers natural-language questions about employee records.

1	Understand the project and architecture
2	Install Python and prepare the workspace
3	Create database, models and schemas
4	Build CRUD and search APIs
5	Build the HTML/JavaScript frontend
6	Integrate the OpenAI Responses API
7	Run and test the complete application
8	Troubleshoot, secure and extend it

Learning outcomes

- Understand frontend, backend, REST API and database responsibilities.
- Create and validate FastAPI endpoints.
- Use SQLAlchemy ORM with SQLite.
- Connect JavaScript to backend APIs using fetch().
- Store API configuration safely in environment variables.
- Send grounded employee context to an AI model and display its response.
- Explain the complete project in an interview.

1. Understand the Application

The application has two major capabilities:

NORMAL FEATURES	Add, view, search, update and delete employee records. These operations use Python and SQL directly.
AI FEATURE	Ask natural-language questions such as 'Who has the highest salary?' The backend supplies employee data as context to the AI model.

Application architecture

Browser	Displays form, table, search and AI question box.
JavaScript	Uses fetch() to send and receive JSON.
FastAPI	Validates requests, runs business logic and returns responses.
SQLAlchemy	Maps Python objects to database tables.
SQLite	Stores employee records in a local file.
OpenAI API	Generates natural-language answers from supplied context.
DATA FLOW	Browser → JavaScript → FastAPI → Validation → Database/AI → JSON response → Browser

Technology stack

Python 3.11+	Programming language
FastAPI	Web API framework
Uvicorn	Development server
Pydantic	Request and response validation
SQLAlchemy	ORM and database access
SQLite	Beginner-friendly database
HTML + Bootstrap	User interface
JavaScript	Frontend logic
OpenAI Python SDK	AI integration

2. Prerequisites and Installation

Install Python from python.org. During Windows installation, select Add Python to PATH.

Verify Python

TERMINAL

```
python --version  
  
# Windows alternative  
py --version
```

Create project folder

TERMINAL

```
mkdir ai-employee-app  
cd ai-employee-app
```

Create and activate a virtual environment

A virtual environment keeps this project's packages separate from other Python projects.

TERMINAL

```
# Windows  
py -m venv venv  
venv\Scripts\activate  
  
# macOS / Linux  
python3 -m venv venv  
source venv/bin/activate
```

Install packages

TERMINAL

```
pip install fastapi uvicorn sqlalchemy openai python-dotenv
```

CHECKPOINT

Your terminal should show (venv). Run `pip list` and confirm that fastapi, uvicorn, sqlalchemy, openai and python-dotenv are installed.

3. Create the Project Structure

PROJECT TREE

```
ai-employee-app/
|-- main.py
|-- database.py
|-- models.py
|-- schemas.py
|-- requirements.txt
|-- .env
|-- .gitignore
`-- static/
    |-- index.html
```

requirements.txt

requirements.txt

```
fastapi
uvicorn
sqlalchemy
openai
python-dotenv
```

.env

.env

```
OPENAI_API_KEY=place_your_api_key_here
OPENAI_MODEL=gpt-5.6
```

.gitignore

.gitignore

```
.env
venv/
__pycache__/
employees.db
```

SECURITY

Never place the API key in HTML or JavaScript. Never publish .env. The official OpenAI quickstart recommends storing the key safely as an environment variable.

MODEL ACCESS

Use a model available to your API project. Keeping the model name in .env lets you change it without editing Python code.

4. Configure the Database

Create database.py. SQLite will automatically create employees.db when the application starts.

database.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

DATABASE_URL = "sqlite:///./employees.db"

engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False}
)

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

Base = declarative_base()

def get_database():
    database = SessionLocal()
    try:
        yield database
    finally:
        database.close()
```

Understand the code

create_engine	Creates the connection between SQLAlchemy and SQLite.
SessionLocal	Creates one database session for each request.
declarative_base	Creates the parent class for ORM models.
yield database	Supplies the session to an endpoint.
finally: close()	Closes the session even if an error occurs.
check_same_thread	Allows SQLite to work with FastAPI request handling.

5. Create the Employee ORM Model

Create models.py. This class maps Python attributes to columns in the employees table.

models.py

```
from sqlalchemy import Column, Float, Integer, String

from database import Base

class Employee(Base):
    __tablename__ = "employees"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False)
    age = Column(Integer, nullable=False)
    salary = Column(Float, nullable=False)
    designation = Column(String(100), nullable=False)
```

Column explanation

id	Integer, primary key and indexed
name	Maximum 100 characters; required
age	Integer; required
salary	Floating-point number; required
designation	Maximum 100 characters; required

ORM IDEA

Instead of manually writing INSERT and SELECT for every operation, we work with Employee objects. SQLAlchemy generates the SQL.

6. Create Pydantic Schemas

Create schemas.py. Pydantic validates incoming JSON before an endpoint executes.

schemas.py

```
from pydantic import BaseModel, ConfigDict, Field

class EmployeeCreate(BaseModel):
    name: str = Field(min_length=3, max_length=100)
    age: int = Field(ge=18, le=60)
    salary: float = Field(gt=0)
    designation: str = Field(min_length=2, max_length=100)

class EmployeeUpdate(EmployeeCreate):
    pass

class EmployeeResponse(EmployeeCreate):
    id: int
    model_config = ConfigDict(from_attributes=True)

class AIQuestion(BaseModel):
    question: str = Field(min_length=3, max_length=500)
```

Why separate schemas?

EmployeeCreate	Validates POST request data.
EmployeeUpdate	Validates PUT request data.
EmployeeResponse	Defines safe output with generated ID.
AIQuestion	Validates an AI question between 3 and 500 characters.

422 ERROR

If data violates a Field rule, FastAPI automatically returns HTTP 422 with validation details.

7. Build the FastAPI Backend

Create main.py. The complete backend is shown below. Type it section by section and test each endpoint in Swagger.

main.py

```
import os
from pathlib import Path

from dotenv import load_dotenv
from fastapi import Depends, FastAPI, HTTPException, Query
from fastapi.responses import FileResponse
from openai import OpenAI
from sqlalchemy import or_
from sqlalchemy.orm import Session

import models
import schemas
from database import engine, get_database

load_dotenv()

app = FastAPI(
    title="AI-Powered Employee Management System",
    version="1.0.0"
)

BASE_DIR = Path(__file__).resolve().parent
models.Base.metadata.create_all(bind=engine)

@app.get("/", include_in_schema=False)
def display_home_page():
    return FileResponse(
        BASE_DIR / "static" / "index.html",
        media_type="text/html"
    )

@app.post(
    "/api/employees",
    response_model=schemas.EmployeeResponse,
    status_code=201
)
def add_employee(
    employee: schemas.EmployeeCreate,
    database: Session = Depends(get_database)
):
    new_employee = models.Employee(**employee.model_dump())
    database.add(new_employee)
    database.commit()
    database.refresh(new_employee)
    return new_employee
```

main.py - continued

```

@app.get(
    "/api/employees",
    response_model=list[schemas.EmployeeResponse]
)
def get_all_employees(
    database: Session = Depends(get_database)
):
    return database.query(models.Employee).all()

@app.get(
    "/api/employees/{employee_id}",
    response_model=schemas.EmployeeResponse
)
def get_employee_by_id(
    employee_id: int,
    database: Session = Depends(get_database)
):
    employee = database.get(models.Employee, employee_id)
    if employee is None:
        raise HTTPException(404, "Employee not found")
    return employee

@app.put(
    "/api/employees/{employee_id}",
    response_model=schemas.EmployeeResponse
)
def update_employee(
    employee_id: int,
    employee_data: schemas.EmployeeUpdate,
    database: Session = Depends(get_database)
):
    employee = database.get(models.Employee, employee_id)
    if employee is None:
        raise HTTPException(404, "Employee not found")

    for field, value in employee_data.model_dump().items():
        setattr(employee, field, value)

    database.commit()
    database.refresh(employee)
    return employee

@app.delete("/api/employees/{employee_id}")
def delete_employee(

```

main.py - continued

```

        employee_id: int,
        database: Session = Depends(get_database)
    ):
        employee = database.get(models.Employee, employee_id)
        if employee is None:
            raise HTTPException(404, "Employee not found")

        database.delete(employee)
        database.commit()
        return {"message": "Employee deleted successfully"}

@app.get(
    "/api/employees-search",
    response_model=list[schemas.EmployeeResponse]
)
def search_employees(
    keyword: str = Query(min_length=1),
    database: Session = Depends(get_database)
):
    search_value = f"%{keyword}%"
    return (
        database.query(models.Employee)
        .filter(
            or_(
                models.Employee.name.ilike(search_value),
                models.Employee.designation.ilike(search_value)
            )
        )
        .all()
    )

@app.post("/api/assistant")
def ask_ai_assistant(
    request: schemas.AIQuestion,
    database: Session = Depends(get_database)
):
    api_key = os.getenv("OPENAI_API_KEY")
    model_name = os.getenv("OPENAI_MODEL", "gpt-5.6")

    if not api_key:
        raise HTTPException(500, "OPENAI_API_KEY is not configured")

    employees = database.query(models.Employee).all()
    employee_context = "
".join(
        f"ID: {e.id}, Name: {e.name}, Age: {e.age}, "

```

main.py - continued

```

        f"Salary: {e.salary}, Designation: {e.designation}"
        for e in employees
    ) or "No employee records are available."

    instructions = """
You are an HR assistant for an Employee Management System.
Answer using only the employee information supplied.
Do not invent employee records.
If information is unavailable, clearly say so.
Use simple, concise and professional English.
"""

    user_input = f"""
Employee information:
{employee_context}

User question:
{request.question}
"""

    try:
        client = OpenAI(api_key=api_key)
        response = client.responses.create(
            model=model_name,
            instructions=instructions,
            input=user_input
        )
        return {"answer": response.output_text}
    except Exception as error:
        print(f"OpenAI error: {error}")
        raise HTTPException(
            status_code=502,
            detail="Unable to get a response from the AI service"
        )

```

8. Understand the Backend Step by Step

Application startup

load_dotenv()	Loads .env values into the Python process.
FastAPI(...)	Creates the web application.
BASE_DIR	Builds a reliable path to index.html.
create_all	Creates missing database tables at startup.
Depends(get_database)	Injects one managed database session.

CRUD flow

POST	Validate request → create Employee → add → commit → refresh → return.
GET all	Query the employees table and return a list.
GET by ID	Look up primary key; return 404 when missing.
PUT	Find record → copy validated fields → commit → refresh.
DELETE	Find record → delete → commit → return confirmation.
SEARCH	Use case-insensitive LIKE on name or designation.

COMMIT VS REFRESH

commit() saves changes. refresh() reloads the object so generated values such as ID are available.

HTTP STATUS

201 means created, 404 means not found, 422 means validation failed, 500 means server configuration error, and 502 means the external AI service failed.

9. Build the Full-Stack Frontend

Create the static folder and then create static/index.html. Bootstrap controls layout; JavaScript connects the page to FastAPI.

static/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>AI Employee Manager</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
  <style>
    body { background:#f4f7fb; }
    .main-title { color:#173b64; font-weight:700; }
    .card { border:0; border-radius:15px; box-shadow:0 5px 18px #00000014; }
    #aiAnswer { min-height:100px; white-space:pre-wrap; background:#eef5ff; }
  </style>
</head>
<body>
<div class="container py-4">
  <h1 class="main-title text-center mb-4">AI-Powered Employee Management System</h1>
  <div id="message" class="alert d-none"></div>

  <div class="row g-4">
    <div class="col-md-4">
      <div class="card p-4">
        <h4 id="formTitle">Add Employee</h4>
        <form id="employeeForm">
          <input type="hidden" id="employeeId">
          <div class="mb-3">
            <label class="form-label">Name</label>
            <input id="name" class="form-control" minlength="3" required>
          </div>
          <div class="mb-3">
            <label class="form-label">Age</label>
            <input id="age" type="number" class="form-control" min="18" max="60" required>
          </div>
          <div class="mb-3">
            <label class="form-label">Salary</label>
            <input id="salary" type="number" class="form-control" min="1" step="0.01" required>
          </div>
          <div class="mb-3">
            <label class="form-label">Designation</label>
            <input id="designation" class="form-control" minlength="2" required>
          </div>
          <button class="btn btn-primary" type="submit">Save Employee</button>
          <button class="btn btn-secondary" type="button" onclick="resetForm()">Clear</button>
        </form>
      </div>
    </div>
    <div class="col-md-8">
```

static/index.html - continued

```

<div class="card p-4">
  <div class="d-flex gap-2 mb-3">
    <input id="searchKeyword" class="form-control" placeholder="Search name or designation">
    <button class="btn btn-success" onclick="searchEmployees()">Search</button>
    <button class="btn btn-outline-secondary" onclick="loadEmployees()">All</button>
  </div>
  <div class="table-responsive">
    <table class="table table-striped">
      <thead class="table-dark">
        <tr><th>ID</th><th>Name</th><th>Age</th><th>Salary</th><th>Designation</th><th>Actions</th></tr>
      </thead>
      <tbody id="employeeTableBody"></tbody>
    </table>
  </div>
</div>
</div>
</div>

<div class="col-12">
  <div class="card p-4">
    <h4>AI HR Assistant</h4>
    <p class="text-muted">Ask questions about employee records.</p>
    <div class="input-group">
      <input id="aiQuestion" class="form-control" placeholder="Who has the highest salary?">
      <button id="askButton" class="btn btn-primary" onclick="askAssistant()">Ask AI</button>
    </div>
    <div id="aiAnswer" class="border rounded p-3 mt-3">AI response will appear here.</div>
  </div>
</div>
</div>
</div>

<script>
const form = document.getElementById("employeeForm");
const idInput = document.getElementById("employeeId");
const nameInput = document.getElementById("name");
const ageInput = document.getElementById("age");
const salaryInput = document.getElementById("salary");
const designationInput = document.getElementById("designation");
const formTitleElement = document.getElementById("formTitle");
const searchInput = document.getElementById("searchKeyword");
const questionInput = document.getElementById("aiQuestion");
const answerBox = document.getElementById("aiAnswer");
const askButtonElement = document.getElementById("askButton");
form.addEventListener("submit", saveEmployee);

function showMessage(text, type="success") {
  const box = document.getElementById("message");
  box.textContent = text;

```

static/index.html - continued

```

    box.className = `alert alert-${type}`;
    setTimeout(() => box.classList.add("d-none"), 3000);
}

async function loadEmployees() {
  try {
    const response = await fetch("/api/employees");
    displayEmployees(await response.json());
  } catch {
    showMessage("Unable to load employees", "danger");
  }
}

function displayEmployees(employees) {
  const tbody = document.getElementById("employeeTableBody");
  if (employees.length === 0) {
    tbody.innerHTML = `<tr><td colspan="6" class="text-center">No employees found</td></tr>`;
    return;
  }
  tbody.innerHTML = employees.map(e => `
    <tr>
      <td>${e.id}</td><td>${escapeHtml(e.name)}</td><td>${e.age}</td>
      <td>Rs. ${Number(e.salary).toFixed(2)}</td><td>${escapeHtml(e.designation)}</td>
      <td>
        <button class="btn btn-warning btn-sm" onclick='editEmployee(${JSON.stringify(e)})'>Edit</button>
        <button class="btn btn-danger btn-sm" onclick="deleteEmployee(${e.id})">Delete</button>
      </td>
    </tr>`).join("");
}

async function saveEmployee(event) {
  event.preventDefault();
  const id = idInput.value;
  const employee = {
    name: nameInput.value.trim(),
    age: Number(ageInput.value),
    salary: Number(salaryInput.value),
    designation: designationInput.value.trim()
  };
  const response = await fetch(id ? `/api/employees/${id}` : "/api/employees", {
    method: id ? "PUT" : "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify(employee)
  });
  const result = await response.json();
  if (!response.ok) return showMessage(result.detail || "Operation failed", "danger");
  showMessage(id ? "Employee updated" : "Employee added");
  resetForm();
}

```


static/index.html - continued

```

    loadEmployees();
}

function editEmployee(e) {
    idInput.value=e.id; nameInput.value=e.name; ageInput.value=e.age;
    salaryInput.value=e.salary; designationInput.value=e.designation;
    formTitleElement.textContent="Update Employee";
    window.scrollTo({top:0, behavior:"smooth"});
}

async function deleteEmployee(id) {
    if (!confirm("Delete this employee?")) return;
    const response = await fetch(`/api/employees/${id}`, {method:"DELETE"});
    const result = await response.json();
    if (!response.ok) return showMessage(result.detail || "Delete failed", "danger");
    showMessage(result.message);
    loadEmployees();
}

async function searchEmployees() {
    const keyword = searchInput.value.trim();
    if (!keyword) return loadEmployees();
    const response = await fetch(`/api/employees-search?keyword=${encodeURIComponent(keyword)}`);
    displayEmployees(await response.json());
}

async function askAssistant() {
    const question = questionInput.value.trim();
    if (question.length < 3) return answerBox.textContent="Enter a valid question.";
    answerBox.textContent="AI is thinking...";
    askButtonElement.disabled=true;
    try {
        const response = await fetch("/api/assistant", {
            method:"POST",
            headers:{"Content-Type":"application/json"},
            body:JSON.stringify({question})
        });
        const result = await response.json();
        answerBox.textContent = response.ok ? result.answer : result.detail;
    } catch {
        answerBox.textContent="Unable to connect to the AI assistant.";
    } finally {
        askButtonElement.disabled=false;
    }
}

function resetForm() {
    form.reset(); idInput.value=""; formTitleElement.textContent="Add Employee";
}

```

static/index.html - continued

```

}

function escapeHtml(value) {
    const div=document.createElement("div"); div.textContent=value; return div.innerHTML;
}

loadEmployees();
</script>
</body>
</html>

```

10. Understand the Frontend

employeeForm	Captures create and update data.
employeeId hidden field	Empty means create; a value means update.
employeeTableBody	Receives generated rows.
loadEmployees	Calls GET /api/employees.
saveEmployee	Chooses POST or PUT and sends JSON.
deleteEmployee	Requests confirmation and calls DELETE.
searchEmployees	Encodes a keyword safely in the URL.
askAssistant	Sends the user's natural-language question.
escapeHtml	Prevents employee text from being treated as HTML.

FETCH PATTERN

fetch(URL, options) → await response.json() → check response.ok → update the page.

TWO VALIDATION LAYERS

HTML validation improves user experience. Pydantic backend validation is still required because frontend checks can be bypassed.

11. Understand the AI Integration

The AI endpoint does not directly query the database by itself. Python first reads employee rows and creates a text context. Then the question and context are sent to the model.

AI FLOW

Question → POST /api/assistant → fetch employees → build context → call Responses API → response.output_text → JSON → page

Core API call

OPENAI PYTHON

```
from openai import OpenAI

client = OpenAI(api_key=api_key)

response = client.responses.create(
    model=model_name,
    instructions=instructions,
    input=user_input
)

answer = response.output_text
```

Prompt components

instructions	Defines the HR role and safety/grounding rules.
employee_context	Contains the records retrieved by Python.
question	Contains the user's request.
model	Selected through OPENAI_MODEL.
output_text	Convenient final text returned by the response.

GROUNDING

The rule 'use only supplied employee information' reduces invention, but important answers must still be verified against the database.

Official reference

The Python SDK installation, environment-variable setup and `client.responses.create(...)` pattern follow the official OpenAI Developer Quickstart: <https://developers.openai.com/api/docs/quickstart>

12. Run the Complete Application

Make sure the virtual environment is active and your terminal is inside ai-employee-app.

TERMINAL

```
uvicorn main:app --reload  
  
# If uvicorn is not recognised  
python -m uvicorn main:app --reload
```

Open these addresses

http://127.0.0.1:8000	Full-stack web page
http://127.0.0.1:8000/docs	Swagger API testing
http://127.0.0.1:8000/redoc	Alternative API documentation

EXPECTED FILE

After the first start, employees.db should appear. This confirms that SQLAlchemy created the SQLite database.

RELOAD

The --reload option automatically restarts the development server when Python files change. Do not use it as a production deployment configuration.

13. Test Every API in Swagger

POST	/api/employees	{"name":"Ravi Kumar","age":28,"salary":65000,"designation":"Java Developer"}
GET	/api/employees	No body
GET	/api/employees/1	No body
PUT	/api/employees/1	{"name":"Ravi Kumar","age":29,"salary":72000,"designation":"Senior Java Developer"}
DELETE	/api/employees/1	No body
GET	/api/employees-search?keyword=developer	No body
POST	/api/assistant	{"question":"Who has the highest salary?"}

Recommended test order

- Add at least three employees.
- Fetch all employees and note their generated IDs.
- Fetch one valid ID and one missing ID.
- Update one record and verify the changes.
- Search by part of a name and designation.
- Ask the AI to compare salaries.
- Delete a record and confirm it is gone.
- Submit invalid age and observe HTTP 422.

14. API and CRUD Revision

Create	POST	/api/employees	201
Read all	GET	/api/employees	200
Read one	GET	/api/employees/{id}	200/404
Update	PUT	/api/employees/{id}	200/404
Delete	DELETE	/api/employees/{id}	200/404
Search	GET	/api/employees-search	200
Ask AI	POST	/api/assistant	200/502

Normal logic versus AI

Exact database operation	Natural-language generation
Predictable result	May vary between calls
Best for filtering and calculations	Best for summaries and explanations
No model usage cost	Uses an external model API
Database is source of truth	Output must be verified

ENGINEERING RULE

Do not use AI for an operation that ordinary Python or SQL can perform more accurately, cheaply and predictably.

15. Common Errors and Solutions

ModuleNotFoundError	Activate venv and run <code>pip install -r requirements.txt</code> .
uvicorn not recognised	Run <code>python -m uvicorn main:app --reload</code> .
404 for index.html	Confirm <code>static/index.html</code> exists and <code>BASE_DIR</code> is correct.
422 validation error	Check name length, age 18-60, positive salary and designation.
OPENAI_API_KEY not configured	Correct <code>.env</code> and restart the server.
401 invalid API key	Use a valid project API key; remove accidental spaces.
Model not available	Set <code>OPENAI_MODEL</code> to a model available to the project.
AI returns 502	Read the server terminal log; check connectivity, account and model access.
Database locked	Stop duplicate processes and ensure sessions close correctly.
Changes not visible	Refresh the browser and check the Network tab/terminal logs.

DEBUG ORDER

Read browser console → inspect Network response → read FastAPI terminal → reproduce in Swagger → isolate database or AI call.

16. Security and Production Notes

This project is designed for learning. Before production deployment, add the following controls:

- Authentication and role-based authorisation.
- Restrictions around salary and personal data.
- Rate limiting for /api/assistant.
- Structured logging without secrets.
- HTTPS and secure deployment configuration.
- Database migrations and PostgreSQL/MySQL.
- Input/output moderation when appropriate.
- AI usage monitoring and budget limits.
- Automated API and frontend tests.
- Backups, audit history and error monitoring.

PRIVACY	Do not send confidential employee data to any external service without organisational approval, a lawful basis and appropriate data-handling controls.
API KEY	The key belongs only on the server. If a key is exposed, revoke it and create a new one; do not merely hide the committed file.

17. Optional: Move from SQLite to MySQL

Install the MySQL driver:

TERMINAL

```
pip install pymysql
```

Change the URL in database.py:

database.py

```
DATABASE_URL = "mysql+pymysql://root:password@localhost/employee_db"
engine = create_engine(DATABASE_URL)
```

Create the database once:

MYSQL

```
CREATE DATABASE employee_db;
```

GOOD PRACTICE

Move the database URL into .env instead of hardcoding the username and password. Use migrations such as Alembic when the schema evolves.

18. Future Enhancements

Beginner	Department field, delete-all endpoint, sorting, pagination, better validation.
Intermediate	JWT login, user roles, PostgreSQL/MySQL, tests, Docker, report export.
AI	Chat memory, structured output, employee-summary generation, policy Q&A.;
Advanced RAG	Upload HR policies, chunk documents, store embeddings, retrieve relevant sections and show sources.
Production	Monitoring, audit logs, background jobs, secrets manager, cloud deployment.

NEXT PROJECT	Convert the AI assistant into an HR policy RAG assistant. Keep employee CRUD exact while retrieving policy content from approved documents.
---------------------	---

19. Interview Explanation

60-SECOND ANSWER

I developed an AI-powered full-stack Employee Management System using Python FastAPI. FastAPI exposes CRUD, search and AI endpoints. Pydantic validates request data, SQLAlchemy manages SQLite records, and a Bootstrap/JavaScript frontend consumes the APIs using fetch. The AI endpoint reads employee records, builds controlled context and calls the OpenAI Responses API from the backend. Configuration is stored in environment variables, and the API key is never exposed to the browser.

Likely interview questions

1. Why did you choose FastAPI?
2. What is dependency injection in this project?
3. What is the difference between Pydantic and SQLAlchemy models?
4. Why do we call commit and refresh?
5. How does the frontend choose POST versus PUT?
6. Why must the AI key stay in the backend?
7. How did you reduce hallucination?
8. Why is SQL better than AI for exact filtering?
9. What would you change for production?
10. How would you add authentication or RAG?

20. Student Assignment and Assessment

Complete the base application first. Then implement the following enhancements independently.

Task 1	Add email and department fields with validation.
Task 2	Add a GET endpoint that filters employees by minimum salary.
Task 3	Add a Delete All button with confirmation and an empty-table error.
Task 4	Ask AI to generate a short professional appreciation message for a selected employee.
Task 5	Add a loading indicator and disable buttons during network requests.
Task 6	Write five valid and five invalid API test cases.
Task 7	Explain which features should use SQL and which should use AI.
Task 8	Present the project in a three-minute demonstration.

Evaluation - 20 marks

Working CRUD	6
Validation and error handling	4
Frontend functionality	4
AI integration and grounding	3
Code clarity and explanation	2
Security awareness	1

21. Final Checklist and Revision Sheet

✓	Python and virtual environment configured.
✓	All packages installed from requirements.txt.
✓	Project structure matches the notes.
✓	Database table created automatically.
✓	POST, GET, PUT, DELETE and search work in Swagger.
✓	Frontend displays records and handles create/update/delete.
✓	.env contains the key and is excluded from Git.
✓	AI endpoint answers using employee context.
✓	Invalid data produces understandable errors.
✓	Student can explain the complete request/response flow.

COMPLETE FLOW

HTML form → JavaScript object → JSON request → Pydantic validation → FastAPI endpoint → SQLAlchemy/AI → JSON response → DOM update

FINAL LEARNING

A full-stack AI application is still a normal software application first. Build reliable data, validation and APIs; add AI only where natural-language intelligence creates real value.

Build it. Test it. Understand it. Then improve it.